Multi-target IC v5: stronger arms, native work priced

Multi-target IC v5 results: the ranking against rho depends on the price of hash and native work

Protocol: `PROTOCOL.md` (frozen in 5aab996e55 before data). Holdout: 33 fresh prime-order curves (group orders about 2^24, 2^28, 2^32), seeds 41 and 42, T in {1024, 4096, 16384}: **3366 runs, 0 errors, every target of every run verified**. Data: `results/holdout/rows.jsonl`, `runs.json` (`mk_runs.py`), `analysis.txt` (primary rule), `analysis_v4rule.txt`, `summary.json`, `summary_v4rule.json`, `noverify.txt` (`noverify.py`). Every number here is [pcc:unverified] until the claim is signed.

Primary rule (section 5a): cost per target in GAE = [(c_I I + M + S)/(c_I + 3) + h (H + L + N)] / T, N = counted native work (rho step-index hashes, distinguished-point canonicalisations and point hashes, IC offset-step indices). icbest = the cheapest of ic5, ic5ks, ic5w, ic5rw per configuration and price; rhobest = the cheapest of the 10 rho runs (rho2xks x 4 kappa, rhodpu x 6). Ratios: medians of 22 per-configuration ratios, bootstrap 95%.

Result

- **Section 5a decisions: icbest vs rhobest unranked in all 27 (cell, c_I)**, and the reason is a clean crossover in h, not noise. At I = 10 M:
  - h = 0: the IC is dearer in all 9 cells, 1.06-1.16x;
  - h = 0.1: close to a tie, 1.00-1.05x (dearer in 8 of 9 cells; 32 b, T = 16384 0.998 [0.992, 1.005]);
  - h = 1: the IC is **cheaper in all 9 cells, 0.71-0.80x** (every 95% upper bound below 1).
  So the multi-target IC is cheaper than the best multi-target rho of this harness when a hash-table or native operation costs about one group addition, and dearer when such operations are nearly free.
- ic5rw vs rhobest: unranked in all 27 as well.
- Under the v4 rule (h on hash-table operations only, N not priced) the IC is dearer in all 27 (1.05-2.02x at I = 10 M).
- Distance to BSGS: icbest is 1.11-1.47x a paired BSGS, dearer in all 27.

Table (I = 10 M, primary rule)

bits	T	icbest/rhobest h=0	h=0.1	h=1	icbest/bsgs2 h=0	GAE/t icbest, rhobest (h=0.1)
24	1024	1.146 [1.138, 1.152]	1.054 [1.046, 1.062]	0.748 [0.742, 0.755]	1.282	158.0, 149.4
24	4096	1.095 [1.089, 1.096]	1.028 [1.022, 1.031]	0.765 [0.761, 0.768]	1.181	95.6, 93.0
24	16384	1.063 [1.060, 1.064]	1.020 [1.016, 1.022]	0.799 [0.796, 0.802]	1.114	65.9, 64.6
28	1024	1.139 [1.131, 1.151]	1.029 [1.020, 1.040]	0.731 [0.724, 0.736]	1.314	508.7, 493.9
28	4096	1.136 [1.127, 1.142]	1.035 [1.029, 1.037]	0.731 [0.724, 0.736]	1.274	276.6, 267.9
28	16384	1.096 [1.090, 1.100]	1.015 [1.010, 1.018]	0.739 [0.735, 0.743]	1.204	157.7, 155.2
32	1024	1.159 [1.143, 1.173]	1.037 [1.024, 1.052]	0.728 [0.717, 0.735]	1.359	1929.8, 1856.1
32	4096	1.130 [1.126, 1.138]	1.014 [1.010, 1.021]	0.717 [0.710, 0.718]	1.306	970.1, 955.4
32	16384	1.101 [1.097, 1.108]	0.998 [0.992, 1.005]	0.713 [0.711, 0.716]	1.261	501.7, 503.8

Without the verification phase that every arm shares (`noverify.txt`, I = 10 M): 1.12-1.21 at h = 0, 1.00-1.07 at h = 0.1, 0.71-0.73 at h = 1.

Which arm wins (per configuration, I = 10 M)

- Rho: `rho2xks` at every price, in all 198 configurations; `rhodpu` never (its two native operations per point outweigh its few hash-table operations once native work is priced). Best kappa: 0.05 (126) or 0.1 (69) at h = 0; 0.1 (111) or 0.05 (63) at h = 0.1; 0.25 (147) or 0.5 (36) at h = 1. **At h = 0 the best kappa is often the grid floor (0.05)**, so a smaller database could make rho cheaper still at h = 0; that can only widen the IC's h = 0 loss. At h = 1 the best kappa is interior, so the IC's h = 1 win does not rest on the grid edge.
- IC: `ic5rw` at h = 0 (198) and h = 0.1 (197); `ic5w` (100) or `ic5` (98) at h = 1. Retention with the random-walk offsets is what makes the IC cheapest when hash operations are free; without hash-heavy retention the IC wins at h = 1.

Why (interpretation)

Both methods store x-classes at one shared inversion per two classes. Per field operation the retention rho covers more of the group than the IC's fixed table, so it wins when only field operations count (h = 0). But rho pays a step-index hash for every step and a hash-table insert or lookup for every point it stores and every walk point it retains, while the IC pays one lookup per trial class and has no per-step native work in its table; at h = 1 that difference dominates and the IC is about 25% cheaper. Neither method uses a property of the curve, and BSGS, which covers the group by tiled windows, remains 11% to 47% cheaper than the IC at every price.

Pre-registered predictions vs outcome

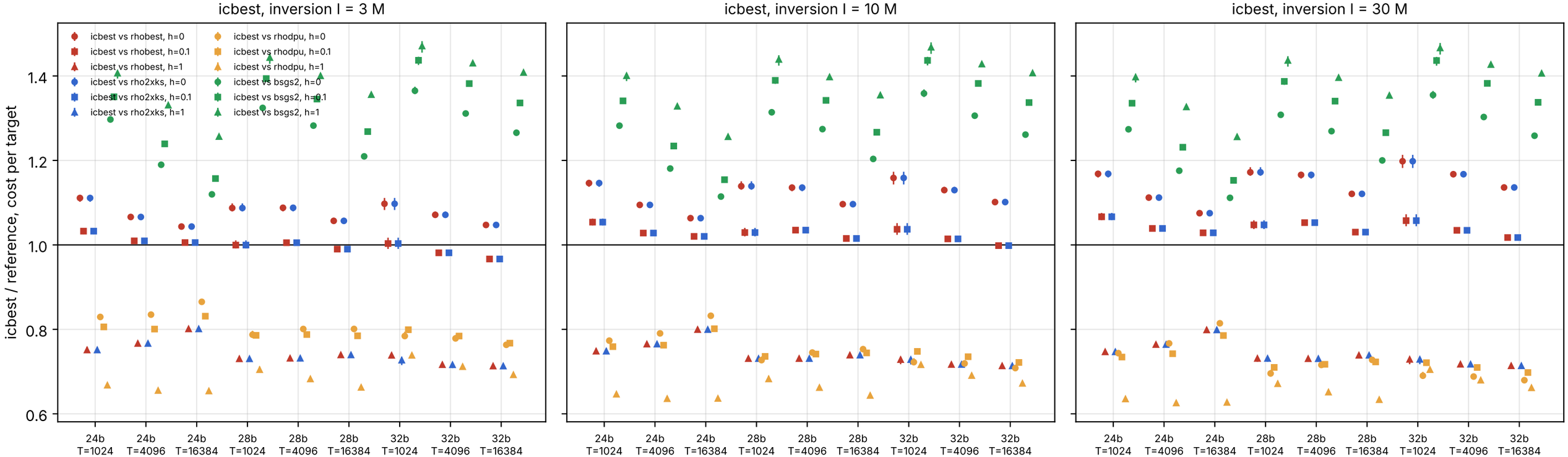
prediction	outcome
rho2xks at small kappa cheapest rho at h = 0 and 0.1; at h = 1 rho2xks or rhodpu	holds (rho2xks at every price)
icbest vs rhobest: rho cheaper at h = 0 (IC about 1.1x), IC cheaper at h = 1 (about 0.7-0.75x), close at h = 0.1; unranked in most or all 27; no cell cheaper	holds:
ic5rw cheapest IC at h = 0; ic5 or ic5w at h = 1	holds
icbest dearer than bsgs2 everywhere	holds

Answer

The multi-target index calculus built here is neither cheaper nor dearer than [pcc:unverified] the best multi-target rho of this harness across the section 5a band: it is 6-16% dearer when hash and native operations are free, about tied at 0.1 GAE per operation, and 20-29% cheaper when they cost one GAE each, in every cell from 2^24 to 2^32 and T = 1024 to 16384. Which one is cheaper in practice is therefore a question of how expensive a hash-table operation is relative to a field multiplication on the target platform, which this counted harness does not measure. It stays 11-47% dearer than BSGS at every price.

Limits

Group orders of about 2^24 to 2^32; costs counted, not timed; memory unbounded for every arm (rho2xks's retained database and the IC's table grow with T); one local host (no AWS identity, so not isolab-eks); the rho arms are this harness's, not ecbench's `rho.multitarget_pooled`; pricing every native operation at the same h as a hash-table operation is the section 5a convention, and the count of rhodpu's native work (2 per point) is a choice that favours rho over the v4 validator's count; the comparison is per target in a batch of T on one curve and says nothing about single-target ECDLP or about ECC2K-130 / m = 83.


icbest, inversion I = 3 M
icbest, inversion I = 10 M
icbest, inversion I = 30 M
icbest / reference, cost per target
1.4
1.2
1.0
0.8
0.6
icbest vs rhobest, h=0
icbest vs rhobest, h=0.1
icbest vs rhobest, h=1
icbest vs rho2xks, h=0
icbest vs rho2xks, h=0.1
icbest vs rho2xks, h=1
icbest vs rhodpu, h=0
icbest vs rhodpu, h=0.1
icbest vs rhodpu, h=1
icbest vs bsgs2, h=0
icbest vs bsgs2, h=0.1
icbest vs bsgs2, h=1
24b T=1024
24b T=4096
24b T=16384
28b T=1024
28b T=4096
28b T=16384
32b T=1024
32b T=4096
32b T=16384